Министерство науки и высшего образования РФ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ЯДЕРНЫЙ УНИВЕРСИТЕТ
«МИФИ»
(НИЯУ МИФИ)

ОТЧЁТ
по теме:
ВЫДЕЛЕНИЕ РЕСУРСА ПАРАЛЛЕЛИЗМА. ТЕХНОЛОГИЯ OPENMP

Студент группы Б24-515 _____ Ясенов Д.М.
подпись, дата

Москва 2026

СОДЕРЖАНИЕ

Введение 3

1 Постановка задачи 4

2 Описание рабочей среды 5

3 Исследование алгоритма 6

3.1 Исходные данные и последовательный алгоритм 6

3.2 Блок-схема последовательного алгоритма 7

3.3 Параллельный алгоритм и OpenMP 7

4 Методика и результаты эксперимента 9

4.1 Время выполнения 10

4.2 Ускорение и эффективность 11

4.3 Сравнение производительности систем 12

4.4 Сводная таблица измерений 14

Заключение 15

ВВЕДЕНИЕ

Цель лабораторной работы — приобрести навыки разработки параллельной программы путём обнаружения ресурса параллелизма в последовательной реализации и его использования средствами OpenMP.

В работе рассматривается поиск элемента с заданным значением в массиве целых чисел. Последовательный алгоритм прекращает просмотр при первом совпадении. В параллельной версии все части массива проверяются независимо, а операция редукции выбирает наименьший индекс подходящего элемента. В отчёте приведены анализ сложности, описание реализации и результаты измерений для 1–16 потоков.

1 Постановка задачи

Требуется исследовать программу поиска заданного значения в массиве и выполнить следующие этапы:

1. описать рабочую среду и используемую версию OpenMP;
2. оценить сложность последовательного поиска в лучшем, среднем и худшем случаях;
3. выявить параллелизм в цикле поиска, реализовать параллельный алгоритм с помощью OpenMP и объяснить выбор директив;
4. построить блок-схему последовательного алгоритма;
5. измерить время работы при различном числе потоков, вычислить ускорение и эффективность, сравнить экспериментальные результаты с моделью Амдала;
6. сформулировать выводы.

2 Описание рабочей среды

Для сравнения сохранены результаты предыдущего запуска на системе, использовавшейся в первой лабораторной работе, и собраны новые измерения на текущем компьютере.

Предыдущая система (Linux):

- **Операционная система:** Kali GNU/Linux Rolling, x86_64.
- **Компилятор:** GCC 13.3.0; **OpenMP:** 4.5 (OpenMP API 201511).
- **Процессор:** Intel Core i5-10300H, 4 физических ядра и 8 логических потоков.
- **Оперативная память:** 3,7 GiB.

Текущая система (macOS):

- **Компьютер:** MacBook Air (Mac17,3).
- **Операционная система:** macOS 26.6 (Tahoe 26.6), архитектура arm64.
- **Процессор:** Apple M5, 10 ядер: 4 производительных и 6 энергоэффективных.
- **Оперативная память:** 16 GB.
- **Компилятор:** GCC 16.2.0 (Homebrew); **OpenMP:** 5.2 (`_OPENMP=202111`), среда выполнения GNU `libgomp`.

Новая версия программы собиралась с флагами `-O2 -fopenmp`. Число потоков задавалось через `omp_set_num_threads`, динамическое изменение числа потоков отключалось через `omp_set_dynamic(0)`. На macOS для GNU `libgomp` привязка потоков недоступна, поэтому в новом запуске использовалось `OMP_PROC_BIND=FALSE`, а `OMP_PLACES` не задавался. Флаги привязки потоков, применявшиеся в прежнем сценарии запуска, для этого измерения не использовались.

3 Исследование алгоритма

3.1 Исходные данные и последовательный алгоритм

Исследуется массив из $N = 10000000$ целых чисел, заполненный результатами генератора псевдослучайных чисел `rand()` с начальным значением 920214. Значение для поиска — 16; программа принудительно записывает его в элемент с индексом $\frac{N}{2}$, поэтому искомое значение гарантированно присутствует. При наличии нескольких совпадений требуется вернуть наименьший индекс.

Последовательная версия проходит элементы слева направо. После каждого сравнения она либо возвращает индекс найденного элемента, либо переходит к следующему элементу; если совпадений нет, возвращается признак отсутствия. В приведённой конфигурации принудительное размещение элемента в середине гарантирует завершение поиска не позднее этого индекса.

В лучшем случае совпадение находится в первом элементе: выполняется одно сравнение, сложность — $O(1)$. В худшем случае совпадение находится в конце массива либо отсутствует: выполняется до N сравнений, сложность — $O(N)$. Для стандартной оценки среднего успешного поиска при равновероятном положении первого вхождения среди N элементов среднее число сравнений равно $\frac{N+1}{2}$, то есть сложность — $O(N)$. В этом эксперименте положение не выбирается равномерно: элемент специально установлен в середину, а перед ним могут встретиться случайные совпадения. Поэтому оценку $\frac{N+1}{2}$ следует понимать как классическую модель среднего случая, а не как точное число сравнений именно для созданного массива.

3.2 Блок-схема последовательного алгоритма

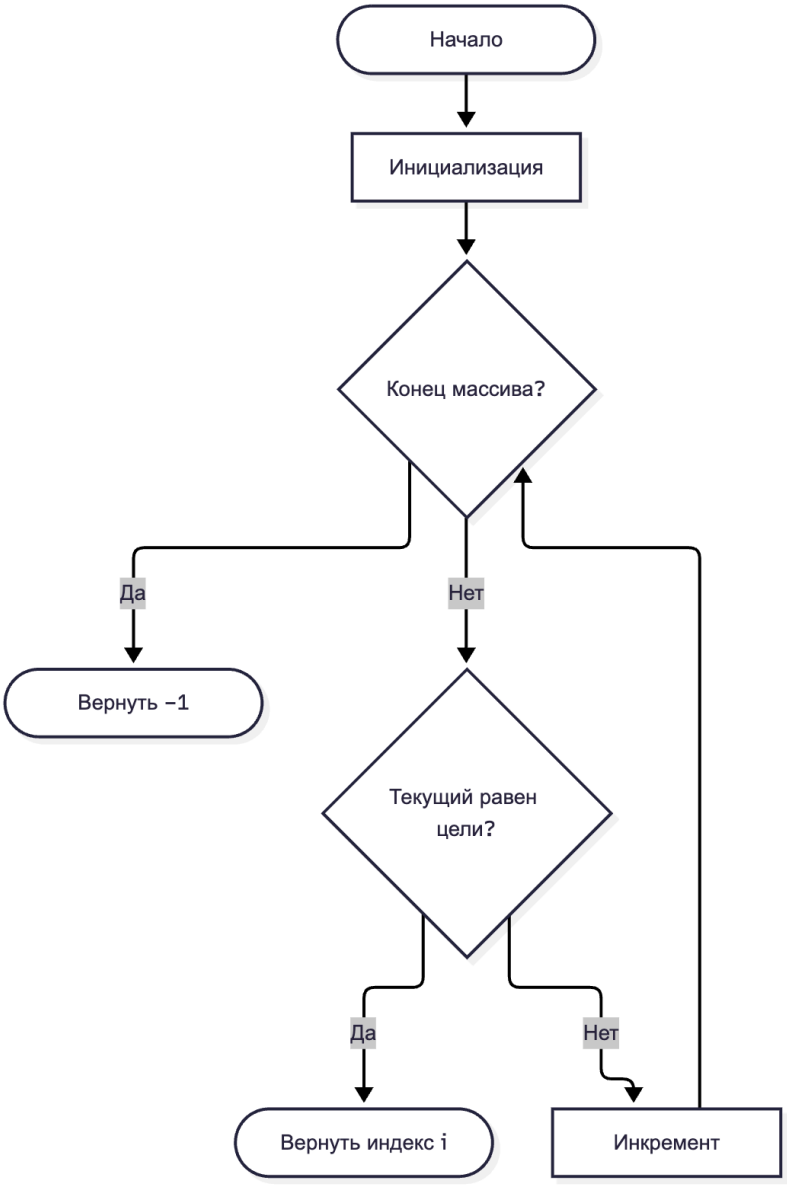


Рисунок 1 — Блок-схема последовательного поиска первого вхождения

На рисунке 1 показано последовательное сравнение элементов массива с искомым значением и завершение при первом совпадении.

3.3 Параллельный алгоритм и OpenMP

Проверки элементов массива независимы: результат сравнения для одного индекса не зависит от результатов для остальных. Поэтому цикл по индексам можно распределить между потоками. В отличие от последовательной версии, для сохранения минимального индекса параллельная версия должна проверить все элементы.

В программе используется `#pragma omp parallel for reduction(min : index)`. Конструкция `parallel for` создаёт команду потоков и распределяет между ними итерации цикла. Каждому потоку доступен общий для чтения массив; переменная `index` участвует в редукции `min`. OpenMP предоставляет потокам приватные промежуточные значения и после цикла объединяет их операцией минимума. Инициализация `index` значением `INT_MAX` служит нейтральным значением: если совпадений нет, программа выводит индекс −1. Таким образом, запись результатов не создаёт гонки данных, а минимальная редукция возвращает именно первое вхождение.

`omp_set_dynamic(0)` запрещает среде выполнения самостоятельно менять число потоков, а `omp_set_num_threads(threads)` задаёт запрошенное число. Перед измерениями выполняются три прогревочных прохода, после чего измеряется цикл поиска; генерация массива в измеренное время не включена.

Время одной итерации алгоритма остаётся линейным по числу проверяемых элементов. При p потоках идеальная вычислительная оценка составляет $O\left(\frac{N}{p} + \log p\right)$ с учётом объединения частичных минимумов. На практике эффективное ускорение ограничивают пропускная способность памяти, накладные расходы создания и синхронизации потоков и конкуренция за вычислительные ресурсы.

4 Методика и результаты эксперимента

Новый эксперимент выполнен 27 октября 2026 года на MacBook Air с Apple M5. Для числа потоков $p = 1, 2, \ldots, 16$ использовался массив размером $N = 10000000$, целевое значение 16 и начальное значение генератора 920214. Для каждого числа потоков собрано 500 измерений в двух проходах — сначала по возрастанию, затем по убыванию числа потоков. Перед измерениями выполнялись три прогревочных запуска. Время измерялось только вокруг цикла поиска, без генерации массива.

Обозначим через T_1 среднее время выполнения с одним потоком, через T_p — среднее время с p потоками. Экспериментальное ускорение вычислялось как $S_p = \frac{T_1}{T_p}$, а эффективность — как $E_p = \frac{S_p}{p}$. Для сравнения использована модель Амдала

$$S(p) = \frac{1}{(1-\alpha)+\frac{\alpha}{p}}, \quad E(p) = \frac{S(p)}{p}.$$

Параметр модели подбирался перебором по новым средним экспериментальным ускорениям для $p = 1\ldots4$. Получено $\alpha = 0,98$ при среднеквадратичном отклонении RMSE = 0,0580. Кривая Амдала служит теоретическим сравнением и не учитывает ограничения пропускной способности памяти.

4.1 Время выполнения

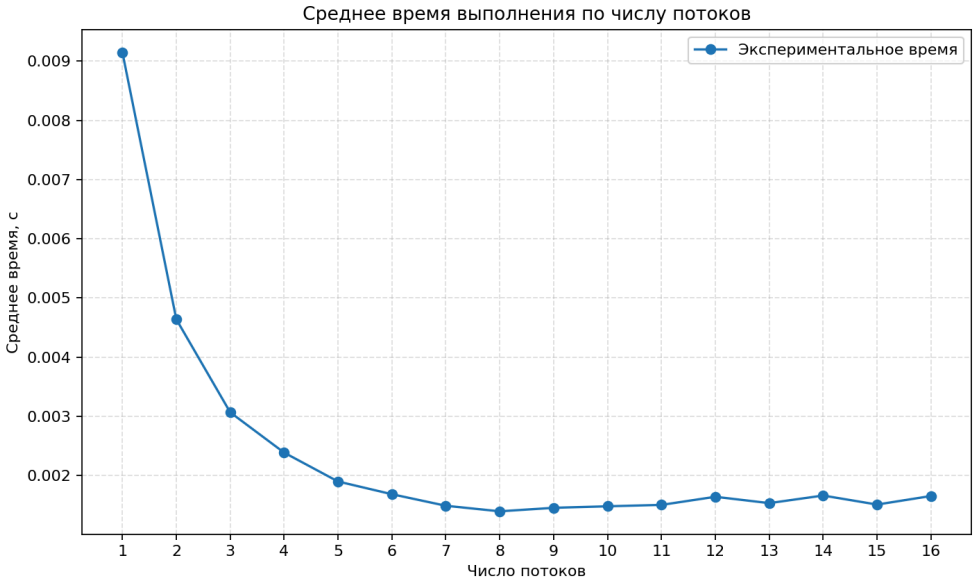


Рисунок 2 — Среднее время поиска на Apple M5 в зависимости от числа потоков

На рисунке 2 показано среднее время нового запуска. При одном потоке оно равно 0,00915 с; наименьшее среднее время — 0,00139 с — получено при восьми потоках. При дальнейшем увеличении числа потоков зависимость становится немонотонной: например, среднее время при 16 потоках составляет 0,00165 с. Результат отражает поведение данной связки процессора, компилятора и OpenMP runtime; доступное число потоков само по себе не гарантирует дальнейшего ускорения.

4.2 Ускорение и эффективность

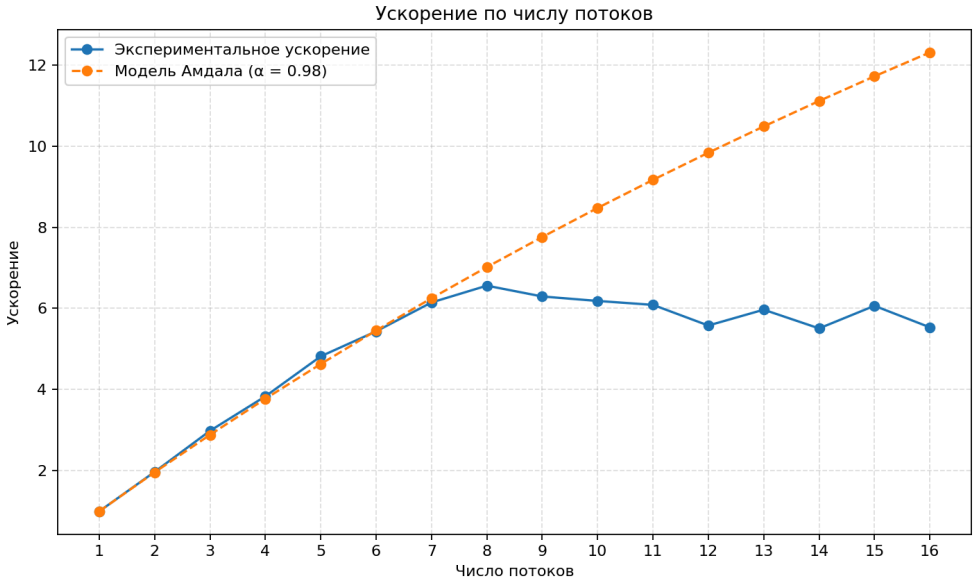


Рисунок 3 — Экспериментальное ускорение и модель Амдала для Apple M5 при $\alpha = 0{,}98$

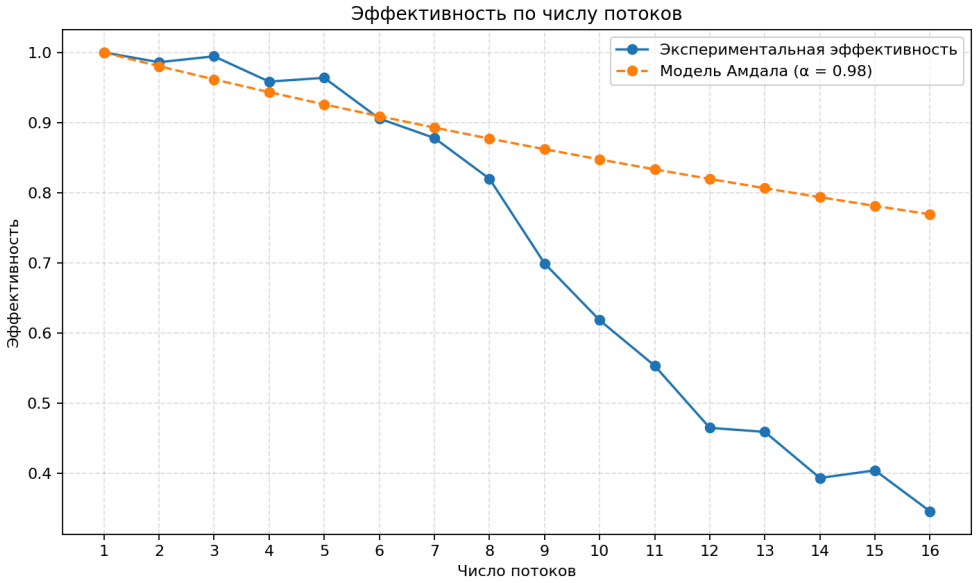


Рисунок 4 — Эффективность и модель Амдала для Apple M5 при $\alpha = 0{,}98$

На рисунке 3 экспериментальное ускорение сравнивается с моделью Амдала, а на рисунке 4 приведены соответствующие значения эффективности. Максимум по среднему времени получен при $p = 8$: ускорение равно 6,560, эффективность — 0,820, или 82,0 %. Для восьми производительных и энергоэффективных ядер процессора такая конфигурация близка к количеству доступных аппаратных ядер. При

дальнейшем увеличении числа потоков эффективность снижается; при $p = 16$ ускорение равно 5,534, а эффективность — 0,346.

Параметр модели Амдала подобран по точкам $p = 1...4$ и составляет $\alpha = 0,98$ (RMSE = 0,0580). Эта оценка описывает только область, использованную для подбора. Её формальная асимптотическая граница $\frac{1}{1-\alpha} = 50$ не является практическим прогнозом: при $p > 8$ модель заметно завышает ускорение и не учитывает ограничения пропускной способности памяти, неоднородность ядер и накладные расходы OpenMP.

4.3 Сравнение производительности систем

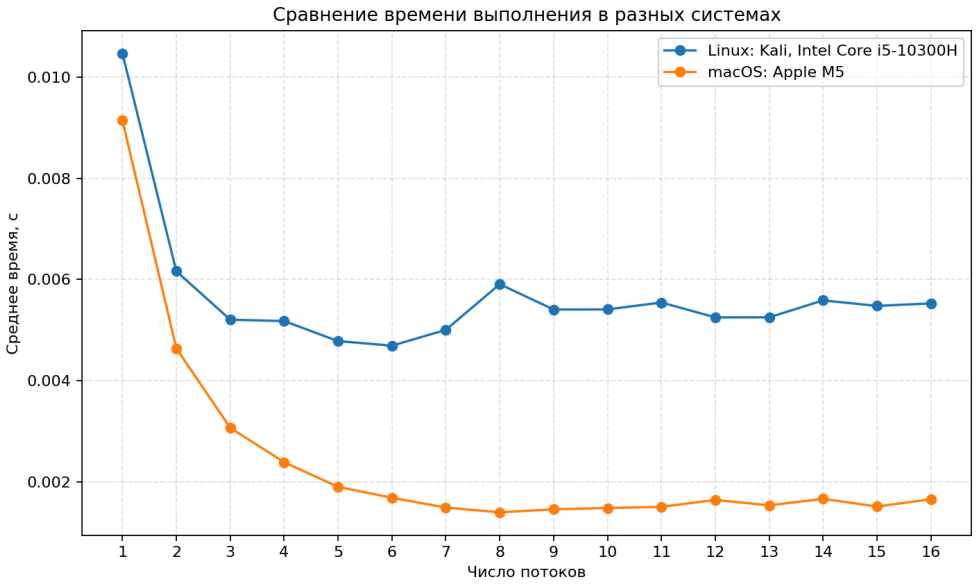


Рисунок 5 — Сравнение среднего времени на Linux и macOS

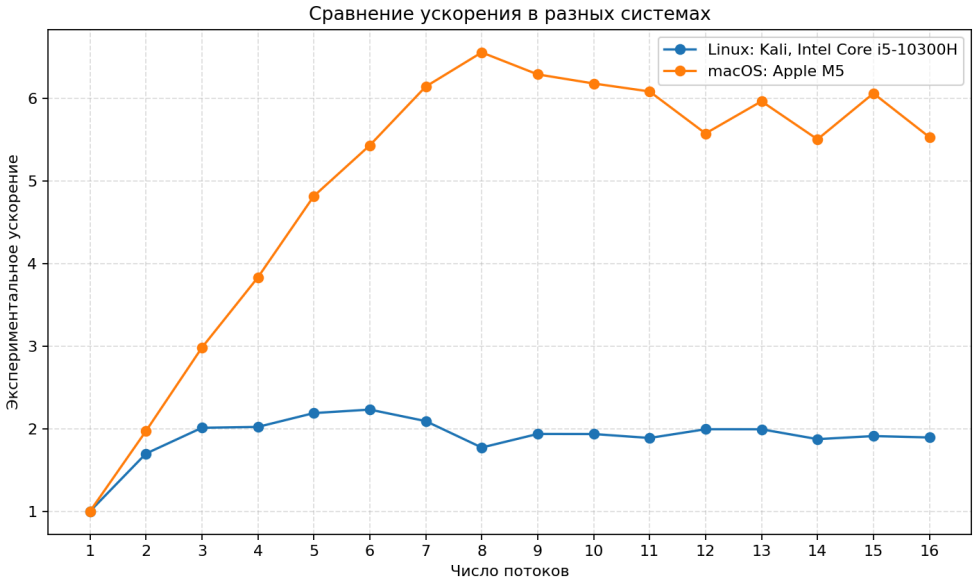


Рисунок 6 — Сравнение ускорения на Linux и macOS

На рисунках 5 и 6 сопоставлены сохранённая сводка прежней системы Kali Linux / Intel Core i5-10300H и новые измерения macOS / Apple M5. Для одного потока среднее время изменилось с 0,01047 с до 0,00915 с. Минимум среди новых средних значений наблюдался при восьми потоках и составил 0,00139 с; ускорение там равно 6,560 против 1,773 при восьми потоках в старой сводке.

Это сравнение относится к двум целым программно-аппаратным платформам, а не только к операционным системам: различаются процессоры, компиляторы и реализации OpenMP. Дополнительно новая система не поддерживает привязку потоков GNU `libgomp`, поэтому она была отключена. Для старого запуска сохранились только агрегированные средние, а не исходные времена отдельных повторов; оценить разброс старых измерений по повторам невозможно.

4.4 Сводная таблица измерений

Таблица 1 — Результаты измерений на Apple M5 и модель Амдала

p	$T_{\text{ср}}$, с	$S_{\text{эксп}}$	$E_{\text{эксп}}$	$S_{\text{Амдал}}$	$E_{\text{Амдал}}$	δ, %
1	0,00915	1,000	1,000	1,000	1,000	0,00
2	0,00464	1,972	0,986	1,961	0,980	0,58
3	0,00307	2,983	0,994	2,885	0,962	3,42
4	0,00239	3,834	0,958	3,774	0,943	1,59
5	0,00190	4,819	0,964	4,630	0,926	4,09
6	0,00168	5,434	0,906	5,455	0,909	0,37
7	0,00149	6,146	0,878	6,250	0,893	1,66
8	0,00139	6,560	0,820	7,018	0,877	6,52
9	0,00145	6,294	0,699	7,759	0,862	18,88
10	0,00148	6,184	0,618	8,475	0,847	27,03
11	0,00150	6,087	0,553	9,167	0,833	33,60
12	0,00164	5,578	0,465	9,836	0,820	43,29
13	0,00153	5,968	0,459	10,484	0,806	43,07
14	0,00166	5,507	0,393	11,111	0,794	50,44
15	0,00151	6,064	0,404	11,719	0,781	48,25
16	0,00165	5,534	0,346	12,308	0,769	55,04

В таблице 1 значение δ — относительное отклонение экспериментального ускорения от теоретического. В области подбора $p = 1…4$ отклонение не превышает 3,42 %. При $p = 16$ расхождение увеличивается до 55,04 %, что подтверждает ограниченность модели Амдала для описания всей кривой ускорения на этой системе.

ЗАКЛЮЧЕНИЕ

В лабораторной работе рассмотрен поиск первого вхождения заданного числа в массиве, определена линейная сложность последовательного алгоритма в среднем и худшем случаях и выделен параллелизм независимых проверок элементов.

Параллельная версия реализована с помощью директивы `#pragma omp parallel for` и редукции `reduction(min : index)`. Редукция позволяет безопасно объединить результаты потоков и сохранить минимальный индекс совпадения. На Apple M5 выполнено по 500 измерений для каждого числа потоков от 1 до 16; исходные времена сохранены для дальнейшего анализа.

Наибольшее ускорение составило 6,560 при восьми потоках. При дальнейшем увеличении числа потоков эффективность снижается, а модель Амдала с $\alpha = 0,98$, подобранным для $p = 1...4$, всё сильнее завышает ускорение. Сводка и графики Linux сохранены отдельно, поэтому исходные файлы прежнего измерения не были перезаписаны. Сравнение показывает различия двух аппаратно-программных платформ, но не позволяет приписать результат исключительно операционной системе: различаются процессоры, компиляторы и поддержка привязки потоков. Новые сырые измерения сохранены в `measurements_macos_m5.csv`, а агрегированные результаты — в `results_macos_m5.csv`.